

Retrieval-Augmented Generation (RAG) Application

A local, document-based question-answering system using PDF ingestion, semantic embeddings, FAISS similarity search, local LLM generation, and RAGAS evaluation.

Project Type	Retrieval-Augmented Generation Application
Answer Generation	Llama 3.2 via Ollama
Evaluation Model	Qwen 2.5 3B via Ollama
Embedding Model	nomic-embed-text
Vector Search	FAISS
Evaluation Framework	RAGAS

Prepared by
Parag Bagga

1. Introduction

This project implements a complete Retrieval-Augmented Generation (RAG) pipeline for answering questions using information contained in PDF documents. Instead of relying only on the internal knowledge of a language model, the application retrieves relevant passages from the supplied documents and provides them as context to the model before generating an answer.

The solution is designed to run primarily with local AI models through Ollama. The pipeline covers document ingestion, text chunking, embedding generation, vector indexing, semantic retrieval, answer generation, short-term conversation memory, and quality evaluation using RAGAS.

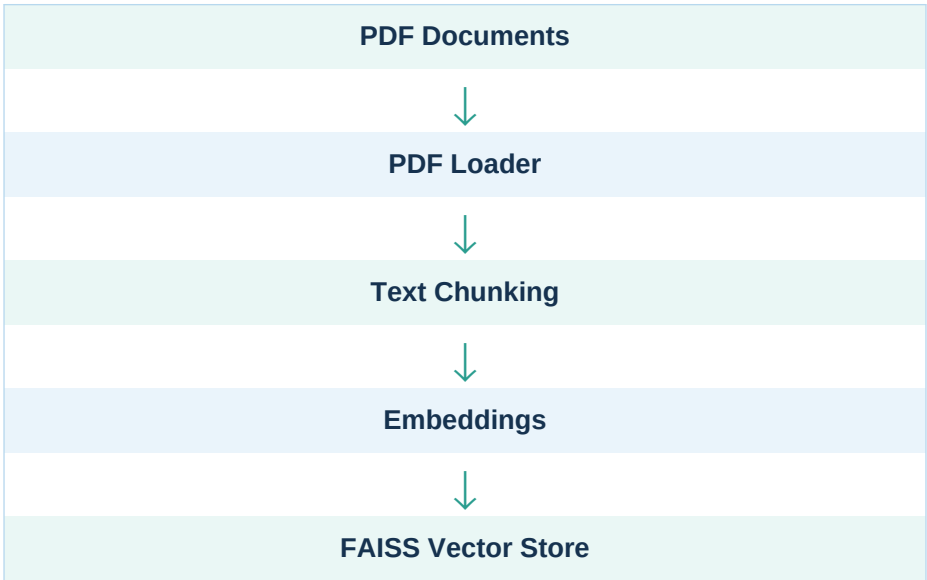
Key design objective: build a modular and reproducible RAG application that can be run locally and evaluated systematically.

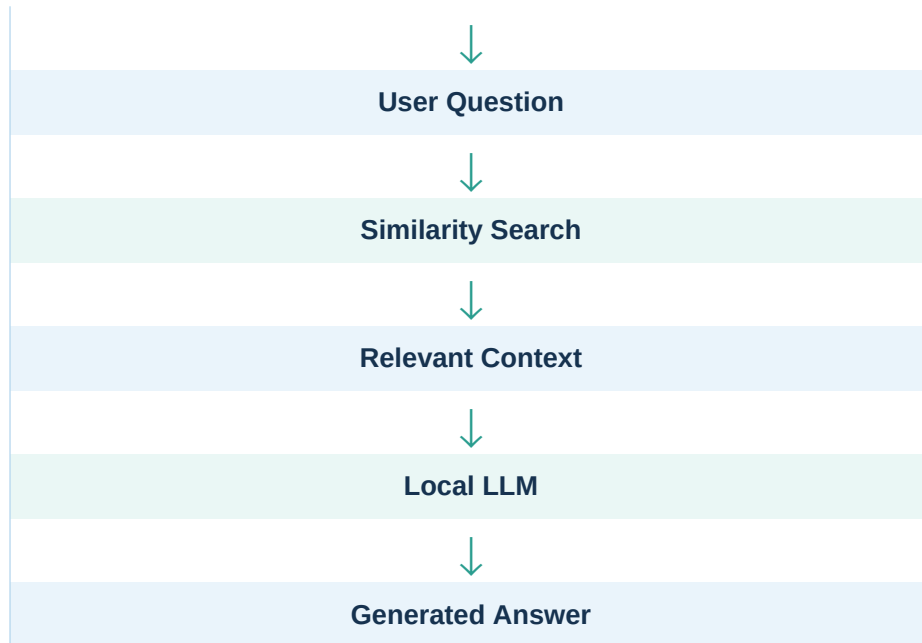
2. Objectives

1. Load and process PDF documents into a searchable knowledge base.
2. Split extracted text into manageable chunks while preserving contextual overlap.
3. Generate semantic embeddings using a local Ollama embedding model.
4. Store vectors in a FAISS index for efficient similarity search.
5. Retrieve the most relevant document chunks for a user question.
6. Generate grounded answers using a local LLM and retrieved context.
7. Support short conversational follow-up through recent conversation memory.
8. Evaluate the RAG pipeline using Faithfulness, Answer Correctness, Context Recall, and Context Precision.

3. System Architecture

The application follows a two-stage architecture: an offline indexing pipeline and an online question-answering pipeline.





4. Technology Stack

Technology	Role in the Project
Python	Core implementation language
Ollama	Local runtime for language and embedding models
Llama 3.2	Generates answers using retrieved document context
Qwen 2.5 3B	Local evaluator model used during RAGAS evaluation
nomic-embed-text	Generates vector embeddings for document text and evaluation
FAISS	Stores vectors and performs similarity search
RAGAS	Evaluates retrieval and answer quality
PyPDF	Extracts text from PDF documents
LangChain	Provides text splitting and model integration utilities
Pandas / Datasets	Stores and processes evaluation results

5. Implementation Details

5.1 PDF Ingestion and Chunking

PDF documents are placed in the data/raw directory. The ingestion component extracts text from the documents and splits it using RecursiveCharacterTextSplitter. The configured values are chunk_size = 1000 and chunk_overlap = 200. The overlap helps preserve information that spans chunk boundaries. The generated chunks are stored in data/processed/chunks.json.

5.2 Embedding Generation

Each text chunk is sent to the local nomic-embed-text model through the Ollama API. The resulting embedding vector is stored together with metadata such as source document, page number, and chunk index. The embedded output is saved as data/processed/embedded_chunks.json.

5.3 FAISS Vector Store

The embedding vectors are converted into a NumPy float32 array and added to a FAISS IndexFlatL2 index. This index enables nearest-neighbour similarity search. The generated index is persisted as data/processed/faiss_index.bin.

5.4 Retrieval

For each user question, the application generates a query embedding and searches the FAISS index for the most similar document chunks. The current configuration retrieves the top 4 results using search(question, top_k=4).

5.5 Answer Generation

The retrieved chunks are formatted into a document context containing source and page metadata. This context, together with the user question and recent conversation history, is sent to the local Llama 3.2 model through Ollama. The prompt instructs the model to use only the provided document context for document-based questions.

5.6 Conversation Memory

The application maintains a short-term memory of recent interactions using `ConversationMemory(max_turns=4)`. This helps the system handle follow-up questions and references to earlier parts of the conversation.

6. Evaluation Methodology

The project uses RAGAS to evaluate both retrieval quality and generated answer quality. Evaluation data is first generated by running predefined questions through the RAG pipeline. The resulting questions, generated answers, retrieved contexts, and reference answers are stored in `data/evaluation_results.json`. RAGAS then evaluates this dataset.

Metric	What It Measures
Faithfulness	Whether the generated answer is supported by the retrieved context.
Answer Correctness	How closely the generated answer matches the reference answer.
Context Recall	Whether the retrieved context contains sufficient information needed to answer the question.
Context Precision	Whether the retrieved chunks are relevant to the question.

7. Evaluation Results

A local evaluation run was completed using a sample of two questions. The evaluator used Qwen 2.5 3B through Ollama, while `nomic-embed-text` was used for embedding-based evaluation components.

Metric	Score	Interpretation
Faithfulness	100.00%	The generated answers were fully supported by the retrieved context in this sample.
Answer Correctness	53.64%	Moderate alignment with the reference answers; this is the main area for improvement.
Context Recall	66.67%	The retrieved context captured a substantial portion of the information required by the questions.
Context Precision	100.00%	The retrieved chunks were highly relevant to the evaluated questions.

Important note: These scores are based on only two evaluated questions and should therefore be treated as an initial validation rather than a statistically comprehensive benchmark. A larger evaluation dataset would provide a more reliable measurement of overall system performance.

8. Analysis and Improvement Opportunities

The results show strong grounding and retrieval precision, while answer correctness and context recall have room for improvement.

Improve Context Recall: Experiment with chunk size and overlap, increase `top_k` carefully, and test retrieval strategies that return complementary rather than highly similar chunks.

Improve Answer Correctness: Refine the generation prompt, use more explicit answer formatting instructions, test stronger local generation models, and ensure retrieved context covers all parts of multi-part questions.

Hybrid Retrieval: Combine vector similarity search with keyword-based retrieval to improve coverage for exact terms and semantic concepts.

Reranking: Use a reranker after initial retrieval to prioritize the most useful chunks before passing context to the LLM.

Larger Evaluation Set: Evaluate more questions covering factual lookup, conceptual questions, multi-step questions, and cases where information is absent.

9. Project Execution Workflow

The complete application can be rebuilt and executed using the following sequence.

Step	Action	Command
1	Install Python dependencies	<code>pip install -r requirements.txt</code>
2	Install local Ollama models	<code>ollama pull llama3.2</code> <code>ollama pull qwen2.5:3b</code> <code>ollama pull nomic-embed-text</code>
3	Process PDF documents	<code>python -m src.ingestion.pdf_loader</code>
4	Generate embeddings	<code>python -m src.ingestion.embeddings</code>
5	Create FAISS index	<code>python -m src.retrieval.vector_store</code>
6	Run the RAG application	<code>python -m src.rag</code>
7	Generate evaluation data	<code>python -m src.evaluation.evaluate</code>
8	Run RAGAS evaluation	<code>python -m src.evaluation.ragas_evaluation</code>

10. Project Structure

```
RAGAssignment/  
├── data/  
│   ├── raw/                # Input PDF documents  
│   ├── processed/          # Chunks, embeddings, FAISS index  
│   ├── evaluation_results.json # Generated evaluation dataset  
│   └── ragas_evaluation_results.csv  
├── src/  
│   ├── ingestion/  
│   │   ├── pdf_loader.py  
│   │   └── embeddings.py  
│   ├── retrieval/  
│   │   ├── vector_store.py  
│   │   └── search.py  
│   ├── generation/  
│   │   ├── llm.py  
│   │   └── memory/  
│   │       ├── conversation_memory.py  
│   ├── evaluation/  
│   │   ├── evaluate.py  
│   │   └── ragas_evaluation.py  
│   └── rag.py  
├── requirements.txt  
└── README.md
```

11. Challenges Encountered

- RAGAS evaluation with external API models encountered token and rate-limit constraints during larger runs.
- Some evaluator generations did not complete when token limits were insufficient.

- Local model evaluation was adopted to reduce dependence on external API quotas.
- RAGAS result objects required careful handling because failed metric evaluations can produce missing values.
- Evaluation speed is influenced by the number of questions, metrics, local model size, and embedding operations.

12. Future Enhancements

- Hybrid search using semantic and keyword retrieval
- Document reranking
- Improved chunking strategies
- Metadata filtering
- Streaming model responses
- Web interface using Streamlit
- Persistent conversation memory
- Larger and more diverse evaluation datasets
- Improved prompt engineering
- Testing alternative embedding and local language models

13. Conclusion

This project demonstrates the complete lifecycle of building a local Retrieval-Augmented Generation application. PDF documents are transformed into searchable chunks, converted into embeddings, indexed using FAISS, and retrieved according to semantic similarity. The retrieved context is then supplied to a local Llama 3.2 model to generate document-grounded answers.

The system was further evaluated using RAGAS with a local Qwen 2.5 3B evaluator. The initial results showed strong faithfulness and context precision, indicating that the retrieved evidence was relevant and that answers were grounded in the supplied context. The lower Answer Correctness and Context Recall scores identify clear opportunities for future improvements in retrieval coverage and answer generation quality.

Overall, the implementation provides a modular, locally runnable, and extensible foundation for document-based question answering using modern RAG techniques.